

Tema 2 (Part 1) – Estructura d'un programa

Seccions 2.1-2.3

Assignatura: Introducció a la programació

Grau Enginyeria Informàtica

Curs 26-27

Inmaculada Rodriguez Santiago

Departament de Matemàtiques i Informàtica



Contingut

- 2.1 Introducció
- 2.2 Estructura general d'un programa
 - 2.2.1 Estructura d'un programa C
 - 2.2.2 Funcions
 - 2.2.3 Programa organitzat en mòduls
- 2.3 Elements del llenguatge C
 - 2.3.1 Variables i constants
 - 2.3.2 Tipus de dades
 - 2.3.3 Expressions i operadors
 - 2.3.4 Àmbit de variables: variables locals i globals
 - 2.3.5 Enums
- 2.4 Estructures de control avançat.
 - Disseny sistemàtic de estructures de control iteratives
 - Concepte de seqüència
 - Esquemes de recorregut i cerca

Tema 2 (Part 1)

Seccions 2.1-2.3

```
_main() {  
      
      
      
      
}
```

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

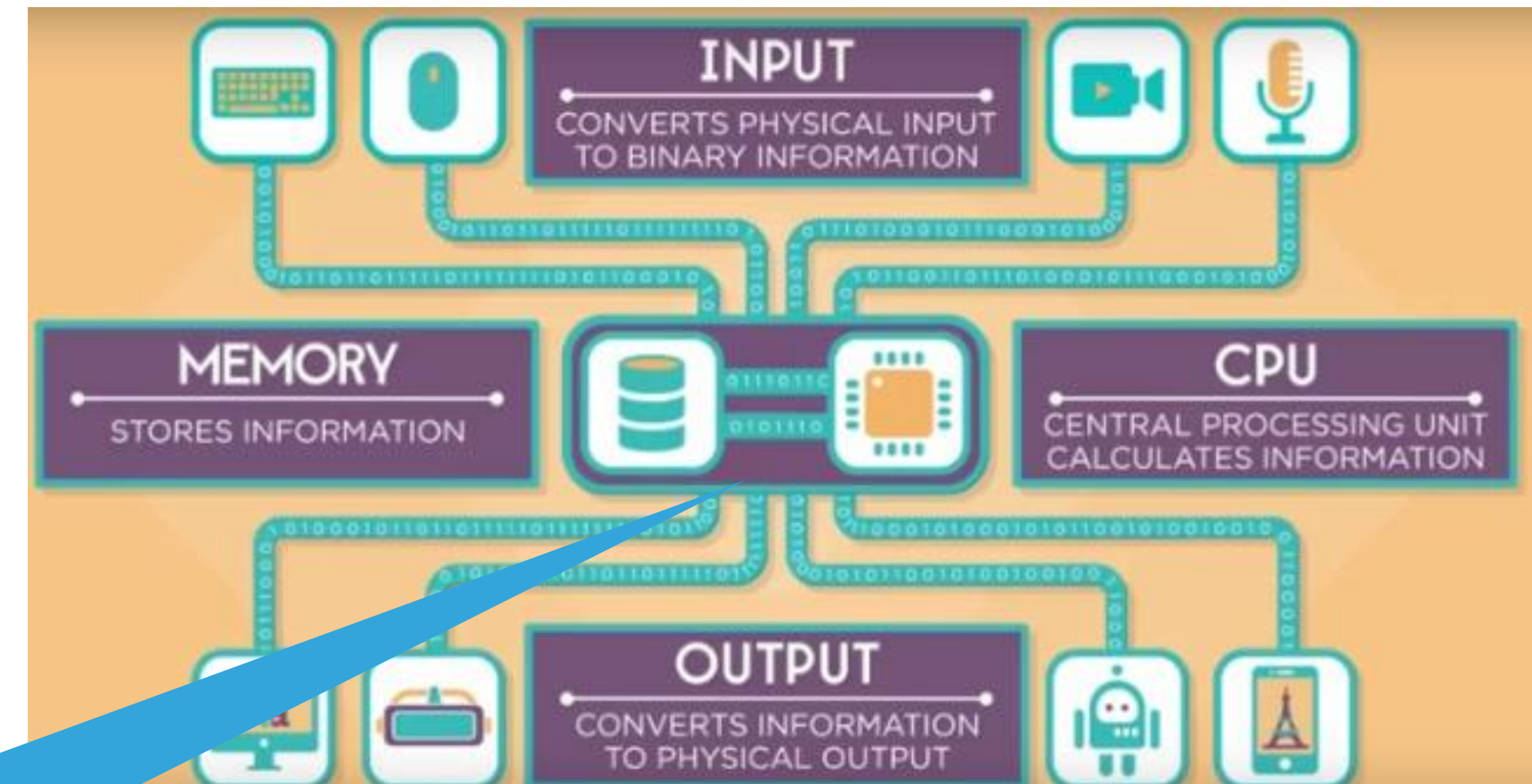
2.1 Introducció

- Recordem el concepte de “programa”

4

Programa: conjunt d'instruccions i dades que poden ser executades per un computador per a solucionar un problema concret.

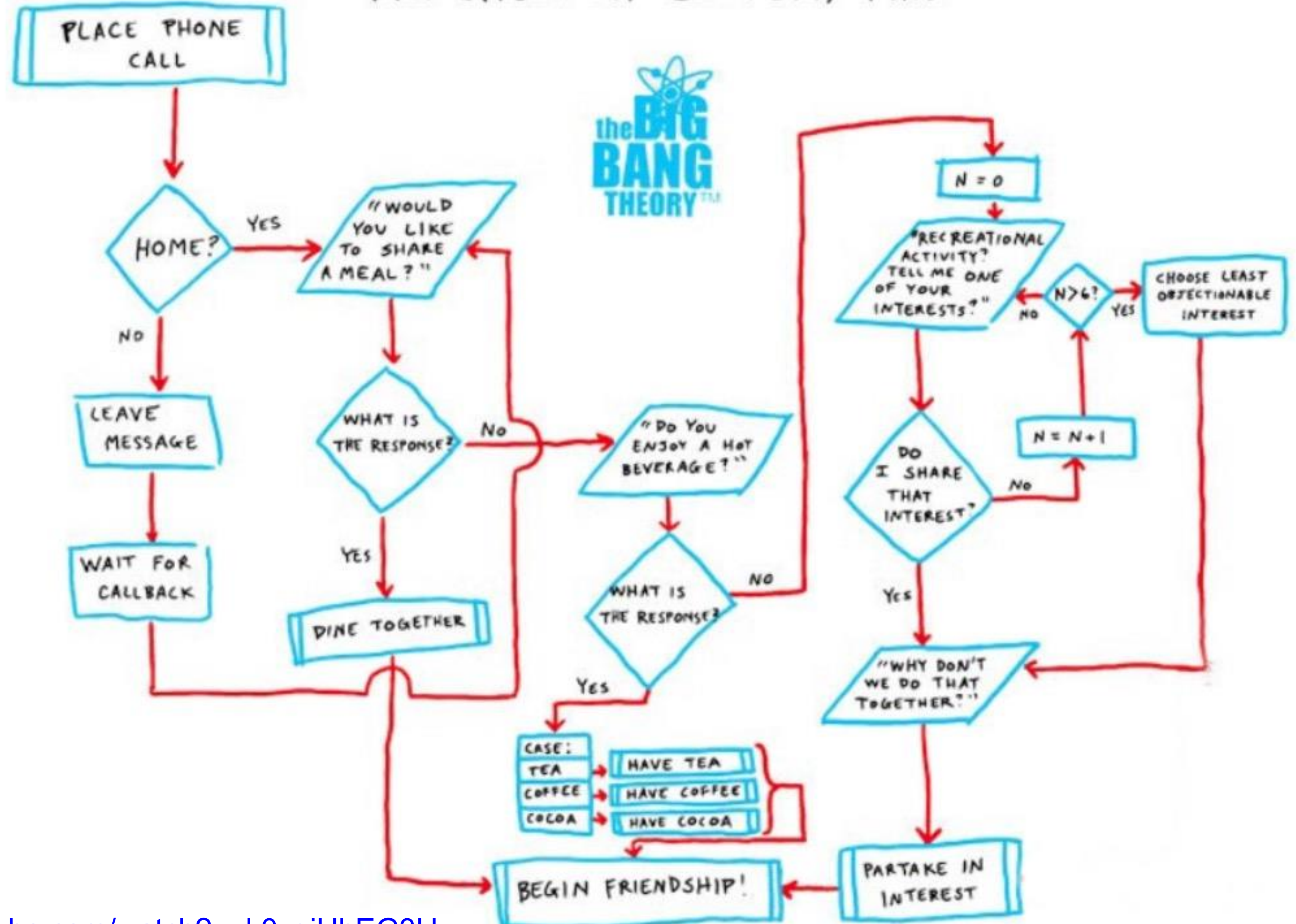
Quan un programa s'executa, les instruccions i les dades estan a la memòria RAM



2.1 Introducció

THE FRIENDSHIP ALGORITHM

DR. SHELDON COOPER, Ph.D



On estan les instruccions i les dades?

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

directives inicials

declaració de funcions

inici funció principal

 instruccions i dades

fi de la funció principal

definició altres funcions (s'utilitzen des de la funció
principal o d'altres)

Java

```
// directives inicials
import java.util.Scanner;

public class Main {

    // altres funcions
    static int suma(int a, int b) {
        return a + b;
    }

    // inici funció principal
    public static void main(String[] args) {
        // dades
        int x = 5;
        int y = 3;
        // instruccions
        int resultat = suma(x, y);
        System.out.println(resultat);
    } // fi funció principal}
```

2.2 Estructura general d'un programa

Python

```
# Directivas iniciales#  
(No necesitamos ninguna)  
  
# Otras funciones  
def suma(a, b):  
    return a + b  
  
#Inicio de la función principal  
def main():  
    # Datos  
    x = 5      y = 3  
    # Instrucciones  
    resultado = suma(x, y)  
    print(resultado)  
# Fin de la función principal  
  
if __name__ == "__main__":  
    main()
```

C

```
// directivas iniciales
#include <stdio.h>

// declaración de función
int suma(int, int);

int main() {
    // dades
    int x = 5;    int y = 3;
    // instrucciones
    int resultat = suma(x, y);
    printf("%d", resultat);
    return 0;
} // fin función principal

// definición de la función
int suma(int a, int b) {
    return a + b;
}
```

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa en C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

```
// directivas iniciales
#include <stdio.h>

// declaración del método
int suma(int, int);

int main() {
    // dades
    int x = 5;    int y = 3;
    // instrucciones
    int resultat = suma(x, y);
    printf("%d", resultat);
    return 0;
} // fin método principal

// definición del método
int suma(int a, int b) {
    return a + b;
}
```

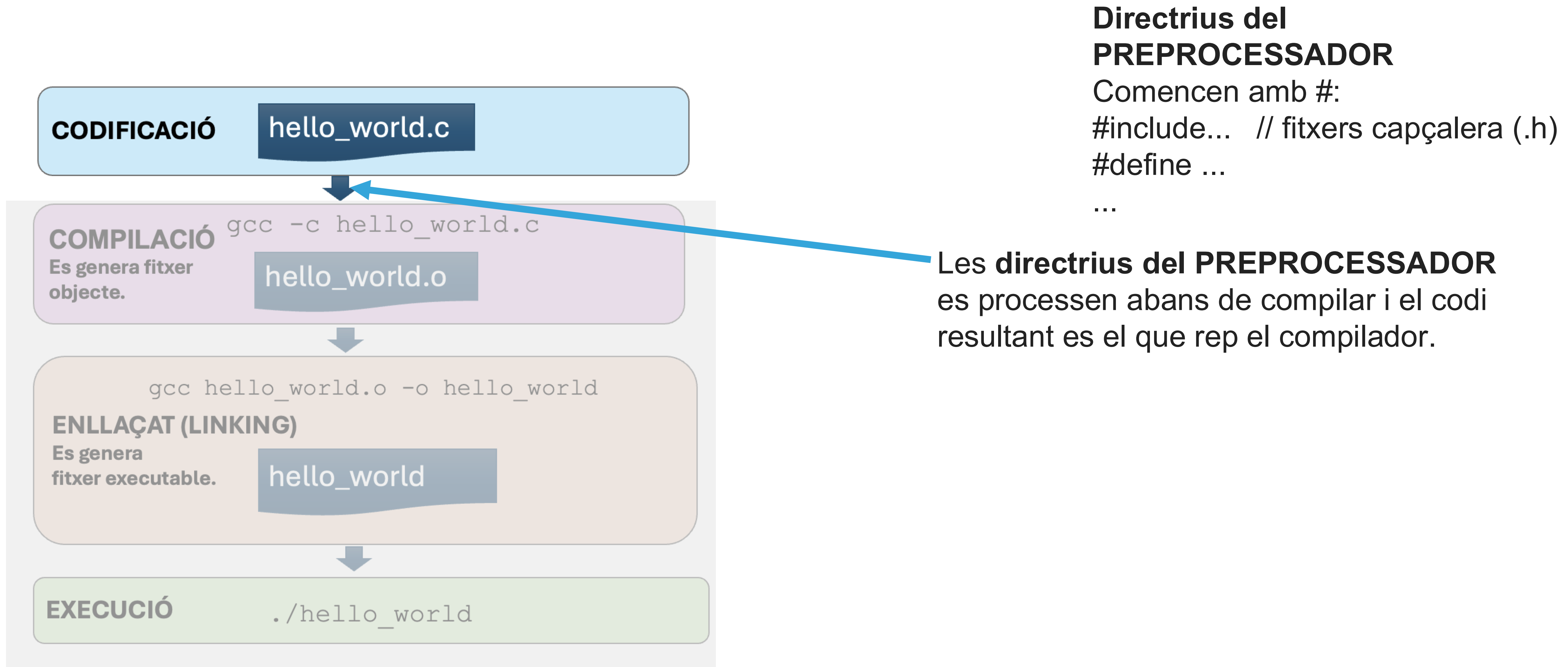
Directrius del PREPROCESSADOR

Comencen amb #:

#include... // fitxers capçalera (.h)

#define ...

...



1

```
#include <stdio.h>
#define MAX 100

int main(void) {

    printf("Hola mundo, MAX = %d\n", MAX);
    return 0;

}
```

hola.c

Directrius del PREPROCESSADOR

`#include`: incorpora el contingut de .h al codi preprocessat

`#define`: substitució de la constant al codi

Directrius del PREPROCESSADOR

`#include`: incorpora el contingut de .h al codi preprocessat

`#define`: substitució de la constant al codi

1

```
#include <stdio.h>
#define MAX 100

int main(void) {

    printf("Hola mundo, MAX = %d\n", MAX);
    return 0;

}
```

hola.c

2

Fem:

```
gcc -E hola.c -o sortida
```

Amb gcc -E podem veure com queda el codi després de ésser processat pel **PREPROCESSADOR**

1

```
#include <stdio.h>
#define MAX 100

int main(void) {

    printf("Hola mundo, MAX = %d\n", MAX);
    return 0;
}
```

hola.c

2

Fem:

```
gcc -E hola.c -o sortida
```

Amb gcc -E podem veure com queda el codi després de ser processat pel **PREPROCESSADOR**

Directrius del PREPROCESSADOR

`#include`: incorpora el contingut de .h al codi preprocessat

`#define`: substitució de la constant al codi

3

Codigo preprocessat (*sortida*)

```
extern int __vsnprintf_chk (char * restrict , size_t ,
                           const char * restrict, va_list);
# 504 "/Library/Developer/CommandLineTools/SDKs/MacOSX10.15.sdk/usr/include/stdio.h"
# 62 "/Library/Developer/CommandLineTools/SDKs/MacOSX10.15.sdk/usr/include/stdio.h"
# 2 "exemple_preprocesador.c" 2

int main()
{
    printf("Hola mundo, MAX = %d\n", 100);
    return 0;
}
```

Aquest codi preprocessat és l'entrada pel compilador

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

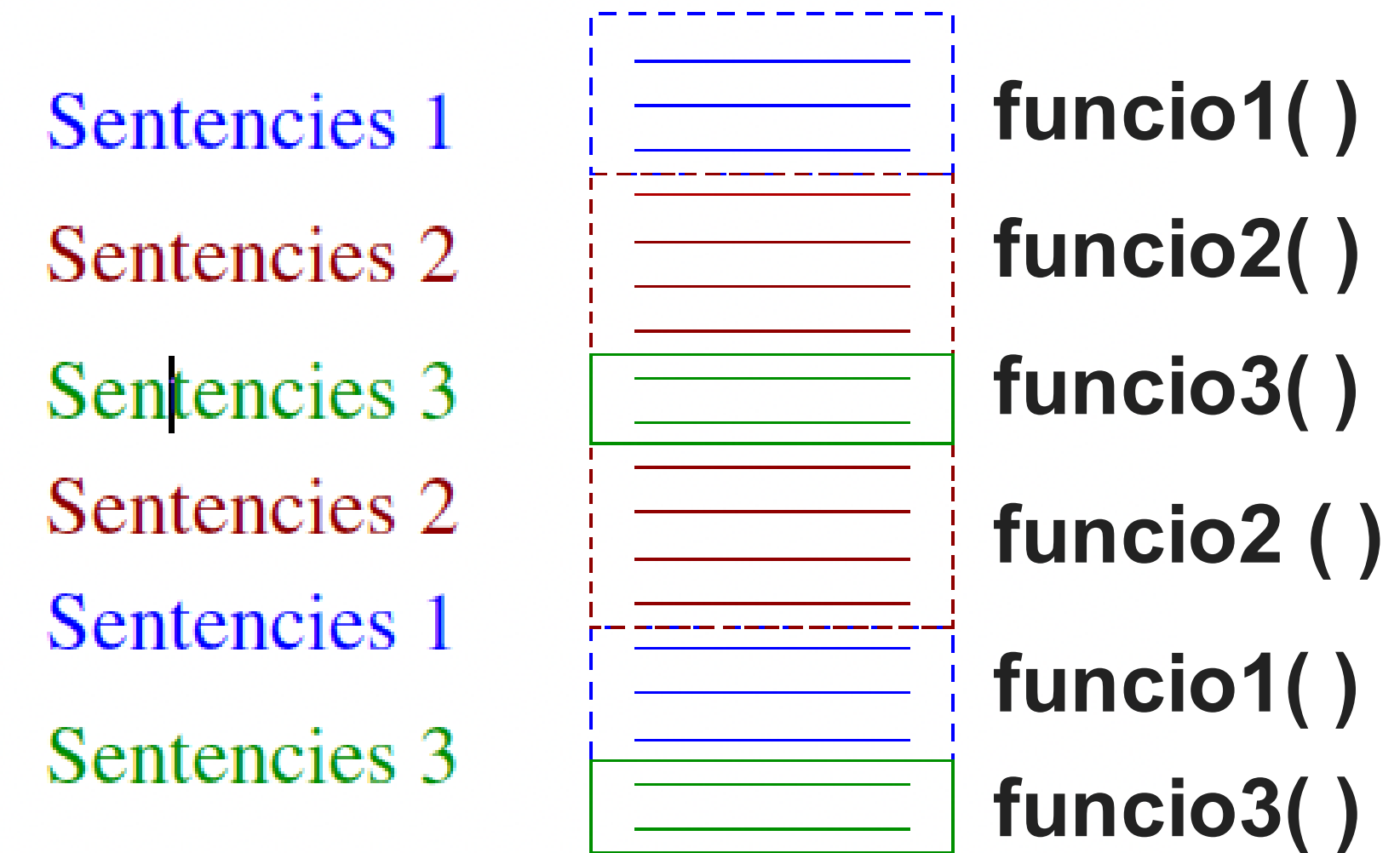
- **Disseny desdendent:** Divisió del problema original en subproblemes més senzills, de forma que solucionant els subproblemes acabarem solucionant el problema original.
- **Programació modular:** divisió del programa en peces més petites, de forma que combinant-les podem resoldre el problema. Aquestes peces són les **funcions**.

- Una funció és un bloc de codi (instruccions i dades) que realitza una tasca determinada.
- Un programa està format per una o més funcions.
- En un programa sempre existeix la funció `main()` per on comença l'execució (i termina) d'un programa.
- Una funció pot cridar altre/s funcions.



Un programa
només pot tenir una
funció `main()` !!

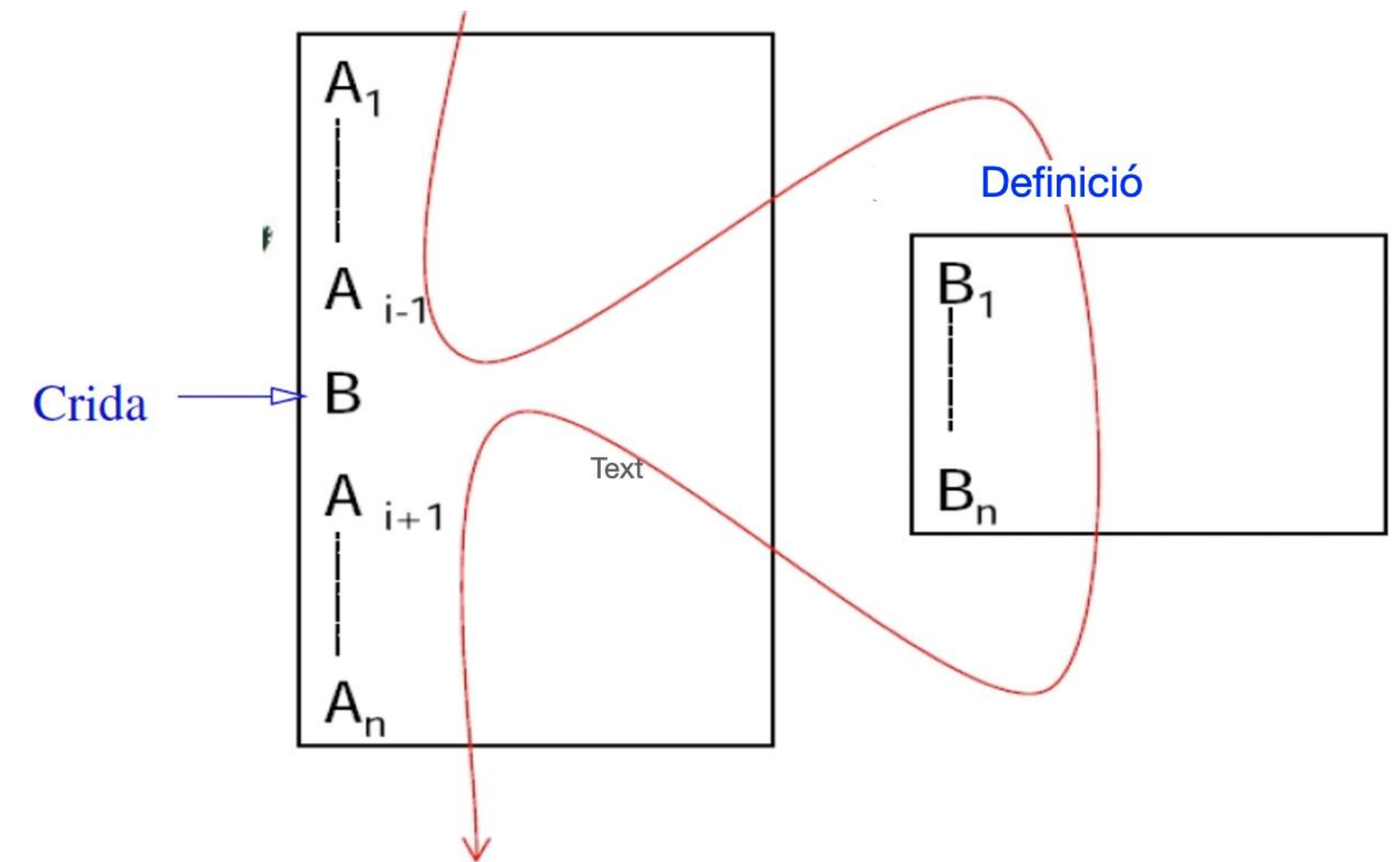
- Evitem la repetició de codi.
Substituïm les línies que es
repeteixen per la crida a la funció



- Avantatges: reutilització, flexibilitat, claredat, robustesa del codi i permet fer proves per separat.

Passos a seguir per a realitzar una funció

- 1) **DECLARACIÓ/CAPÇALERA** de la funció: indica el nom, paràmetres que rep i retorn.
- 2) **DEFINICIÓ/IMPLEMENTACIÓ** de la funció: conjunt d'instruccions i dades que la defineixen.
- 3) **CRIDA a la funció**: s'utilitza la funció. Per exemple: quan des del `main()` fem `suma(3, 4)`.



La **crida** porta el flux d'execució del programa a la definició de la funció.

Quan la funció acaba, el flux retorna a la funció des d'on s'ha fet la crida.

- **Paràmetres FORMALS:** són els paràmetres de la **DEFINICIÓ** de la funció. Per cada paràmetre s'indica el `<tipus> <id_param>`. Si no en té paràmetres s'indica `void`.
- **Paràmetres ACTUALS:** són els paràmetres de la **CRIDA** a la funció. Són els valors que es passen a la funció.
- **Retorn:** a la definició s'indica el tipus de retorn (`void` si no retorna cap valor). Si retorna algun valor, a la crida s'ha de recollir aquest valor.

- **Paràmetres FORMALS:** són els paràmetres de la **DEFINICIÓ** de la funció. Per cada paràmetre s'indica el `<tipus> <id_param>`. Si no en té paràmetres s'indica `void`.
- **Paràmetres ACTUALS:** són els paràmetres de la **CRIDA** a la funció. Són els valors que es passen a la funció.
- **Retorn:** a la definició s'indica el tipus de retorn (`void` si no retorna cap valor). Si retorna algun valor, a la crida s'ha de recollir aquest valor.

S'ha de garantir que:

- Coincideixi el nombre de p. actuals i el nombre de p. formals.
- Coincideixi en ordre, el tipus i el significat dels p. formals i dels p. actuals.

```
int suma (int a, int b);  
int main(void) {  
    int x = 9, y = 4;  
    printf("%d\n", suma( x , y) );  
}  
int suma (int a, int b) {  
    return a+b;  
}
```

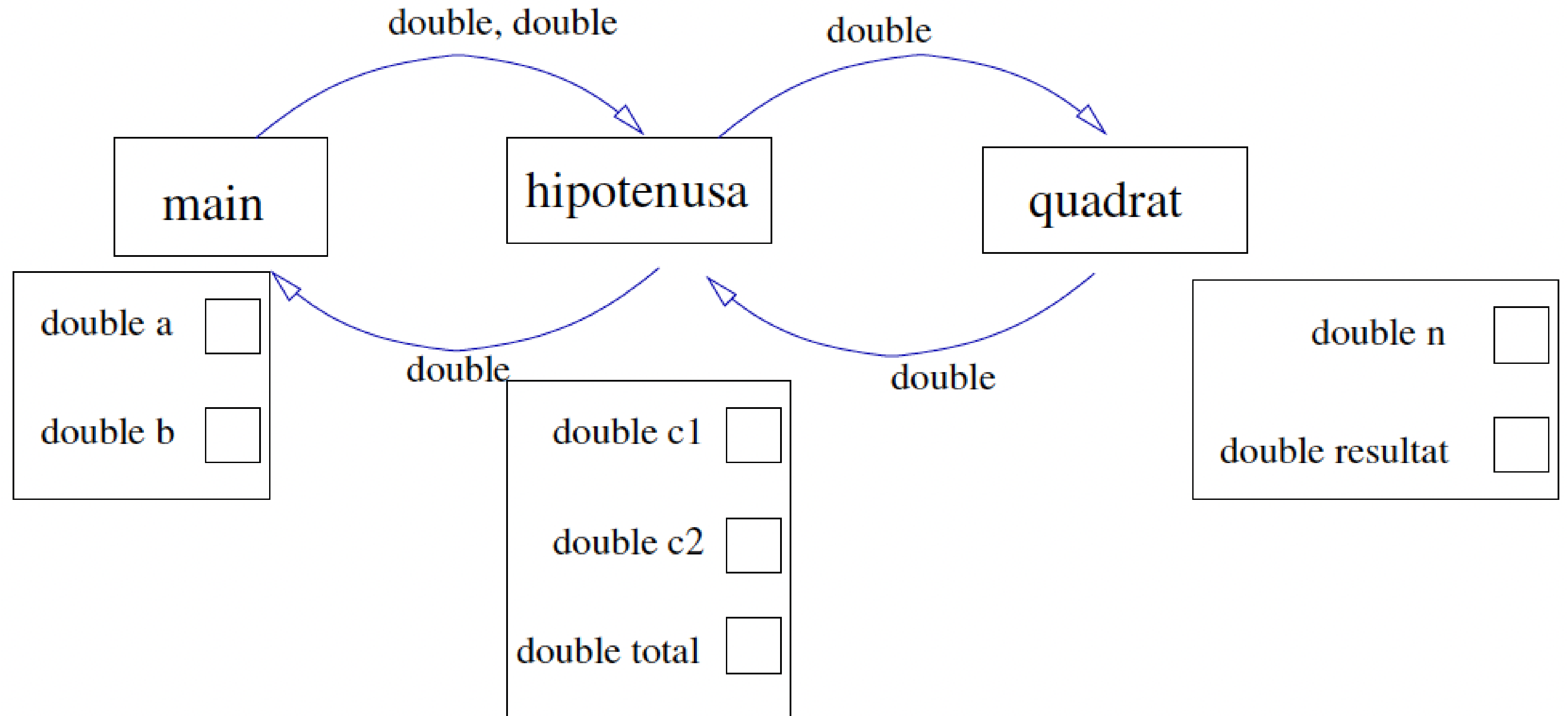
A la crida P. actuals: x i y

Tipus de retorn

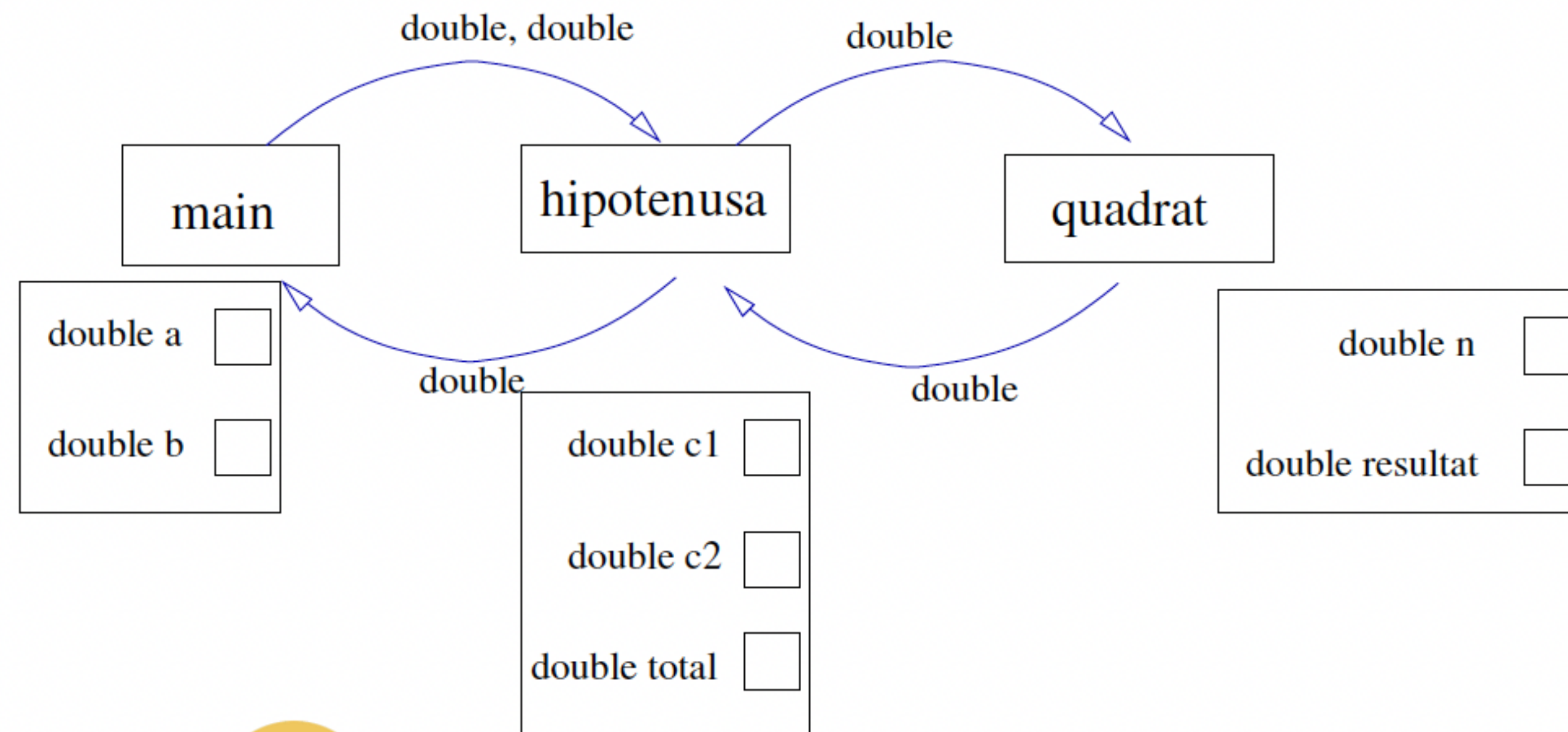
A la definició P. formals: a i b

Des de les funcions fem crides a altres funcions. Les funcions es comuniquen mitjançant paràmetres i valors de retorn.

Des de `main()`
cridem a
`hipotenusa()`, i
des de
`hipotenusa()` a
`quadrat()`.



2.2.2 Funcions



`assert()` serveix per comprovar que una funció retorna el valor esperat!!

```
assert(funcio(param,...,param)==resultat_esperat);
```

```
#include <stdio.h>
#include <math.h>
#include <assert.h>

double hipotenusa(double c1, double c2);
double quadrat(double n);

int main(void) {
    assert(quadrat(3.0) == 9.0);
    assert(quadrat(-3.0) == 9.0);
    assert(quadrat(0.0) == 0.0);
    assert(hipotenusa(3.0, 4.0) == 5.0);

    printf("TOTS els tests han passat\n");
    return 0;
}

double hipotenusa(double c1, double c2) {
    return sqrt(quadrat(c1) + quadrat(c2));
}

double quadrat(double n) {
    return n * n;
}
```

C ens permet usar constants i fer operacions matemàtiques cridant a funcions

```
#include <stdio.h>
#include <math.h>    // per a constants i funcions matemàtiques abs() amb decimals
#include <stdlib.h>  // abs() amb int

int main(void) {
    // constant M_PI de <math.h>
    double pi = M_PI;
    double num = 5.678; int a = -3;

    printf("Constant PI: %f\n", pi);
    printf("Constant PI amb dos decimals: %.2f\n", pi);
    printf("Seno de PI/2: %f | Coseno de PI: %f\n", sin(pi / 2), cos(pi));
    printf("Numero decimal: %f | RedondeoArriba: %.0f\n", num, ceil(num));
    printf("RedondeoAbajo: %.0f | RedondeoCercano: %.0f\n", floor(num), round(num));
    printf("Raiz cuadrada de 16: %f\n", sqrt(16));
    printf("Potencia 2 elevado a 3: %f\n", pow(2, 3)); printf("Valor absolut: %d", abs(a));

    return 0;
}
```

Sortida a
pantalla

```
Constant PI: 3.141593
Constant PI amb dos decimals: 3.14
Seno de PI/2: 1.000000 | Coseno de PI: -1.000000
Numero decimal: 5.678000 | RedondeoArriba: 6
RedondeoAbajo: 5 | RedondeoCercano: 6
Raiz cuadrada de 16: 4.000000
Potencia 2 elevado a 3: 8.000000
Valor absolut: 3
```

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

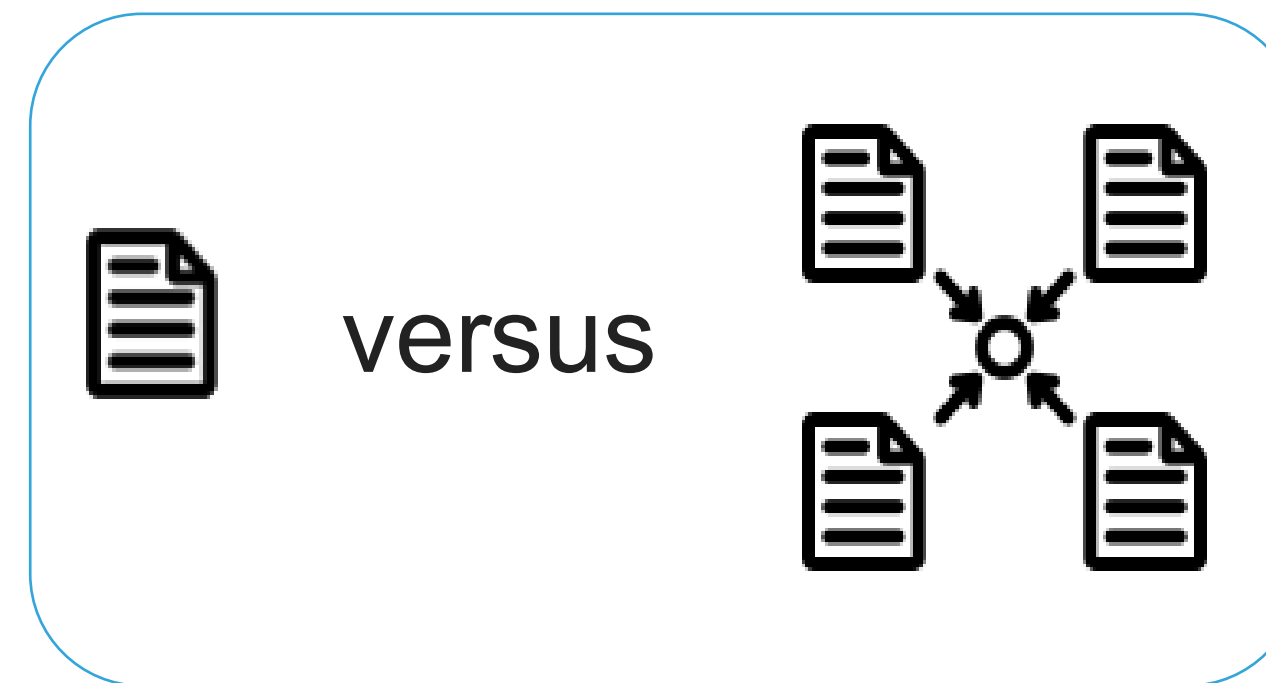
2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca



Font: flaticon

*També pots revisar secció 8 del doc
“Primers passos ..” al CV.*

2.2.3 Programa organitzat en mòduls



Un mòdul agrupa fitxers (amb funcions) per realitzar una tasca determinada.

Avantatges

- El codi és més fàcil de llegir.
- Cada fitxer té una única responsabilitat.
- Les funcions es poden reutilitzar en altres programs (fent `#include` dels `.h`).
- Diversos programadors poden treballar sobre fitxers diferents.
- Els projectes grans són més fàcils de mantenir.

També pots revisar secció 8 del doc “Primers passos ..” al CV.

2.2.3 Programa organitzat en mòduls



Un mòdul agrupa fitxers (amb funcions) per realitzar una tasca determinada.

Avantatges

- El codi és més fàcil de llegir.
- Cada fitxer té una única responsabilitat.
- Les funcions es poden reutilitzar en altres programs (fent `#include` dels `.h`).
- Diversos programadors poden treballar sobre fitxers diferents.
- Els projectes grans són més fàcils de mantenir.

Exemples:

jugador.c, jugador.h
enemics.c, enemics.h
puntuacio.c, puntuacio.h
main_joc.c

funcions.c, funcions.h
main.c

Nota: Afegim també el fitxer amb el `main()` que usa les funcions.



2.2.3 Programa organitzat en mòduls

Exemple: ***funcions.h***, *funcions.c*, *main.c*

```
#ifndef FUNCIONS_H
#define FUNCIONS_H
void funcio_1(void) ;
void funcio_2(void) ;
#endif
```

funcions.h



Els fitxers .h contenen capçaleres/declaracions de funcions, constants, etc., però **NO la implementació/definició** de les funcions.

Sense fitxers .h, hauríem de repetir les declaracions de les funcions en cada fitxer .c on s'utilitzessin aquestes funcions.



2.2.3 Programa organitzat en mòduls

Exemple: *funcions.h*, *funcions.c*, ***main.c***

```
main.c
#include <stdio.h>
#include "funcions.h"

int main(void) {

    int a = 2;
    printf("\nExecutant-se
           main(), a val: %d\n", a);

    assert (funcio_1()==1);
    // ...

return 0;
}
```

Recorda que abans de compilar, el preprocessador “processa” totes les directives #...

- Amb <>, #include <stdio.h>, busca en les carpetes del sistema.
- Amb “”, #include "funcions.h", busca en el directori del projecte.

2.2.3 Programa organitzat en mòduls



33

Exemple: *funcions.h*, ***funcions.c***, *main.c*

```
#include <stdio.h>
#include "funcions.h"

int funcio_1(void) {
    printf("\nExecutant-se la funció 1\n");
    funcio_2();
    return 1;
}

int funcio_2(void) {
    printf("\nExecutant-se la funció 2\n");
    return 2;
}
```

funcions.c

2.2.3 Programa organitzat en mòduls



34

Exemple: *funcions.h*, *funcions.c*, *main.c*

```
#include <stdio.h>
#include "funcions.h"

int main(void) {

    int a = 2;
    printf("\nExecutant-se
           main(), a val: %d\n", a);

    funcio_1();
    // ...

return 0;
}
```

main.c

```
#ifndef FUNCIONS_H
#define FUNCIONS_H
void funcio_1(void);
void funcio_2(void);
#endif
```

funcions.h

```
#include <stdio.h>
#include "funcions.h"

int funcio_1(void) {
    printf("\nExecutant-se la funció 1\n");
    funcio_2();
    return 1;
}

int funcio_2(void) {
    printf("\nExecutant-se la funció 2\n");
    return 2;
}
```

funcions.c

Exercici



Crea una funció `maxim` que rep dos nombres enters i retorne el major dels dos.

```
int maxim(int, int)
```

Organitza el codi en mòduls *maxim.h* *màxim.c* *main.c*.

Exercici



Crea una funció `maxim` que rep dos nombres enters i retorne el major dels dos.

```
int maxim(int, int)
```

Organitza el codi en mòduls *maxim.h* *màxim.c* *main.c*.

Per seguir la tècnica **TDD** (Test-Driven-Development, veure P1 al CV), fes:

- 1) Escriu la **capçalera/declaració** de la funció `maxim()`
- 2) Escriu el `main()` amb els asserts que comproven **què** ha de fer la funció. Per allò, els asserts **criden** la funció.

```
assert(funcio(param,...,param)==resultat_esperat);
```

- 3) Pensa **com** ha de calcular el màxim. Escriu la **implementació/definició** la funció `maxim()`

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

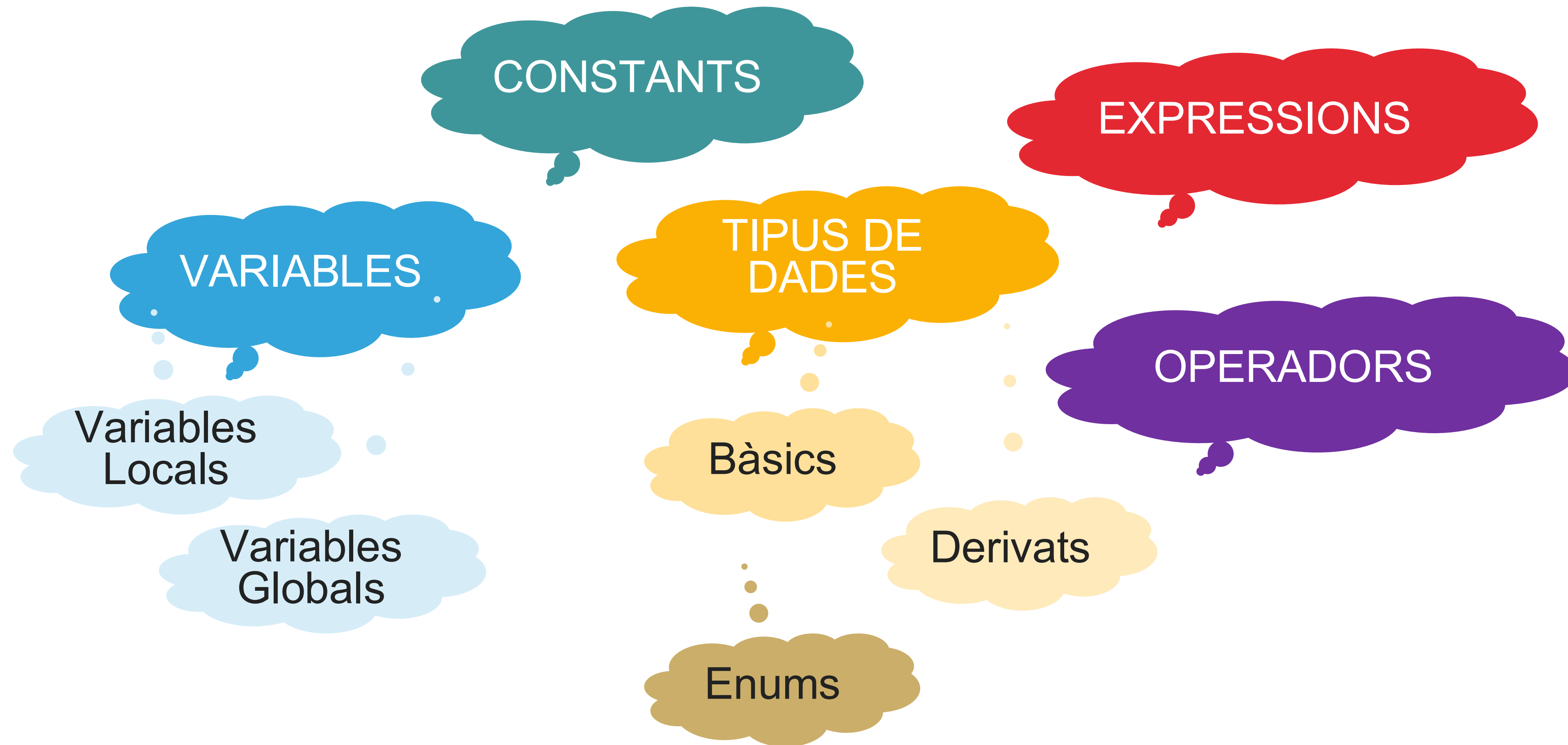
2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca



Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

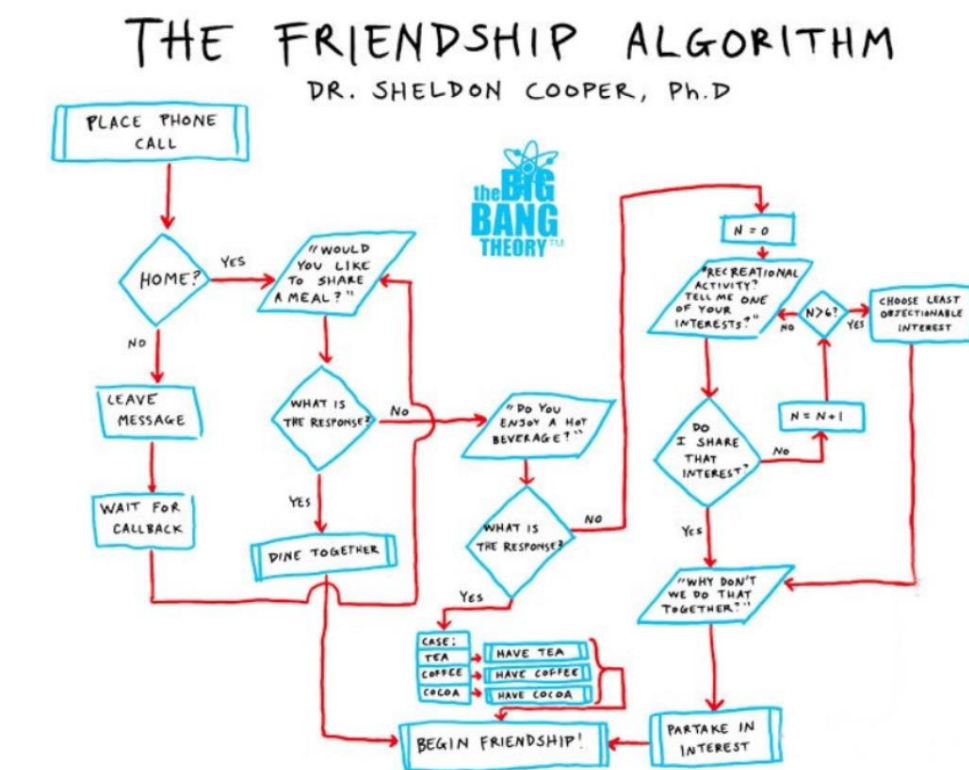
2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

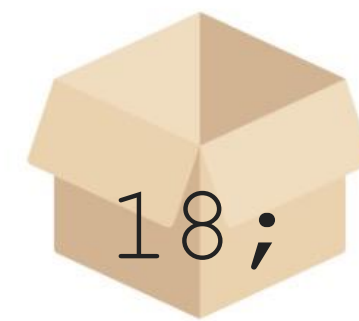
- Per a implementar l'algorisme "Friendship", i altres algorismes, necessitem "caixes" on emmagatzemar dades.
- Les variables i les constants permeten representar les dades (valors) que necessita un programa per resoldre un problema concret.
 - Variables (el **valor canvia** durant l'execució del programa)
 - Constants (el **valor NO canvia** durant l'execució del programa)



- Una **variable** és com una “caixa que guarda coses”. Una variable s'utilitza per emmagatzemar **dades** d'un tipus particular (enters, decimals, cadenes de caràcters, ...).
- Quantitat de memòria que ocupa la variable (grandària de la caixa) depèn del tipus de dada

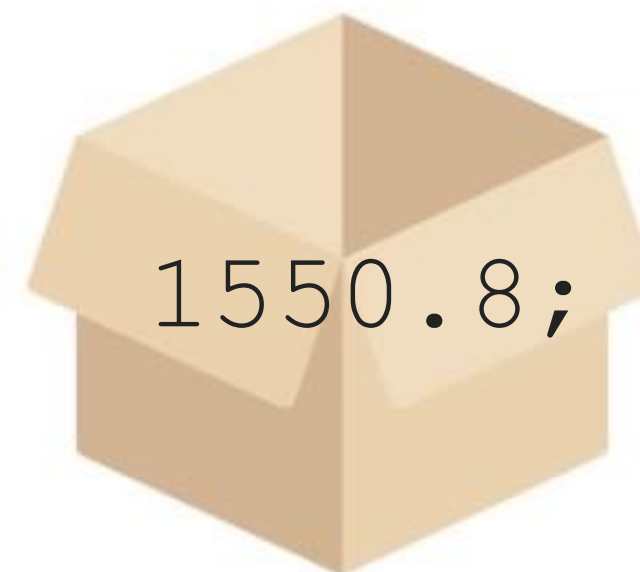
Ex :

short edat =



18;

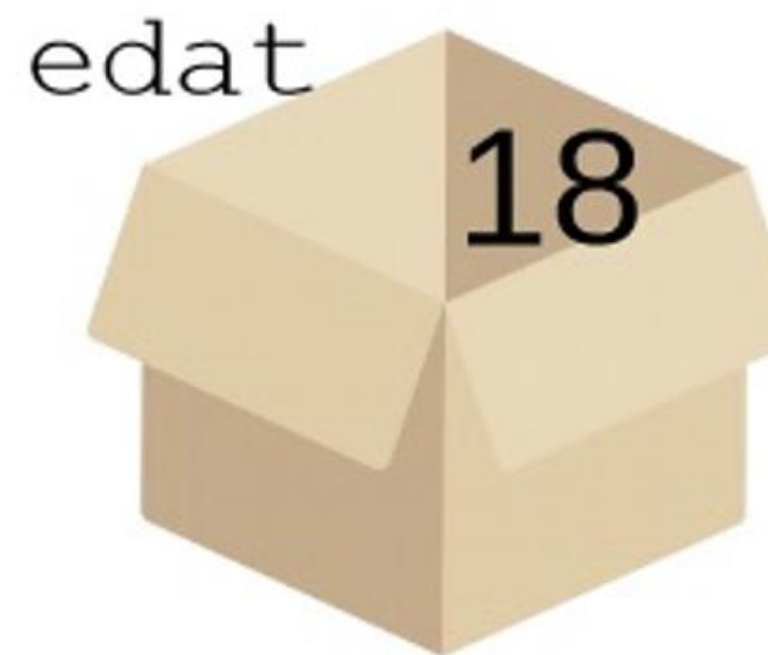
float sou =



1550.8;

Les variables i les constants tenen un **tipus** (determina el conjunt de valors que pot tenir i operacions que es poden realitzar), un **nom** i un **valor**.

```
int edat = 18;
```

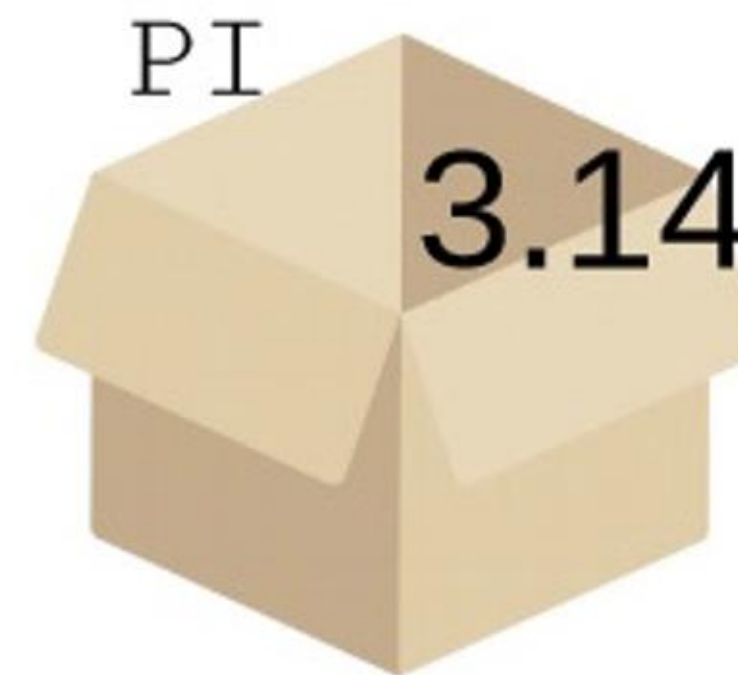


Nom: edat
Valor: 18
Tipus: int

En C, els noms de les variables en minúscula

Noms compostos: snake case
ex. nom_alumne

```
const double PI = 3.14;
```



Nom: PI
Valor: 3.14
Tipus: double

En C, els noms de les constants en majúscula

Declaració de variables a C

```
tipusVariable nomVariable [ = valorInicial];
```

Els [] indiquen que és opcional

- Una variable només es visible a l'àmbit o bloc on està definida.
- Per tant, només es pot utilitzar al bloc on està definida.
- Un bloc es defineix amb { }.

= és l'operador d'assignació
l-value = r-value;
Emmagatzema el valor de la dreta a la variable de l'esquerra.

```
#include <stdbool.h>
int main(void) {
    char car; b = 23;
    bool esMajor = false;
    {
        int a = 55; printf("%d", a);
    }
    car = 'c'; b = 44; esMajor = true;
}
```

2.3.1 Variables i constants

Declaració de constants a C

#define

```
#include <stdio.h>
#define MAX 100
int main()
{
    int a=2, b=4, r;
    if (a < MAX)
        .....
MAX = 25;
    .....
}
```

El preprocessador substitueix totes les ocurrències de MAX per 100 abans de compilar

const

```
#include <stdio.h>
int main()
{
    const int max = 100;
    int a=2, b=4, r;
    if (a < max)
        .....
max = 25;
    .....
}
```

Ocupa un espai de memòria ja que és una variable de tipus `int` que no es pot modificar

- **Regles** per definir els noms de variables i constants en **C**:
- Comença per una lletra o un subratllat (_). Els següents caràcters poden ser lletres, dígitos o subratllats.
- Es distingeixen minúscules i majúscules.
- No es poden usar blancs entremig ni caràcters especial com - + . etc
- No hi ha longitud màxima (però hauria de ser raonable).
- No poden ser paraules clau de C (keywords): if, return, while, etc.

Correctes:

```
id  
nom_usuari  
MAX_LONG
```

Incorrectes:

```
79id  
int  
nom usuari
```

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

Sistemes de tipatge als llenguatges de programació

- statically typed (tipat estàtic): el tipus de les entitats es coneix en temps de compilació (ex. Java)

```
int x = 10;
System.out.println(x + 5)
```
- dynamically typed (tipat dinàmic): el tipus de les entitats es coneix en temps d'execució (ex. Python)

```
x = 10;
print(x + 5)
```
- strongly typed (tipat fort): les entitats es vinculen a un tipus específic, surten errors si operen variables de diferent tipus (ex. Java, python)

```
int x = 10; boolean b = true;
System.out.println(x + b); //error
if (x) ... // error
```
- weakly typed (tipat feble): les entitats també tenen un tipus però si fem operacions amb entitats de diferent tipus, les restriccions son menors (ex. C)

```
int x = 10;
if (x){ // correcte: x es !=0 -> true
...
}
```

<https://medium.com/android-news/magic-lies-here-statically-typed-vs-dynamically-typed-languages-d151c7f95e2b>

- **Bàsics/Primitius:** La mida bàsics depèn de la implementació de C i de la plataforma. Els valors següents són els habituals en ordinadors actuals.

- Enters

`char` (1 byte), `short` (2 bytes), `int` (4 bytes), `long` (4 o 8 bytes), `long long` (8bytes)

Especificadors de tipus: `unsigned`, `signed` (per defecte).

Per exemple: `unsigned int`

- Reals

`float` (4 bytes) (precisió simple 7 dígits)

`double` (8 bytes) (precisió doble 16 dígits), `long double` (10, 12 o 16 bytes)

- Boolean

`bool` (1 byte) es necessita `#include <stdbool.h>`

- Buit/sense valor

`void`

- **Derivats:** Enums, estructures, unions, arrays, punters, punters a funcions.

Tipus	Ocupació (bytes)	Codificació	Rang
char / signed char	1	Complement a 2 / ASCII	−128 ... 127
unsigned char	1	Binari natural	0 ... 255
short	2	Complement a 2	−32768 ... 32767
int	4	Complement a 2	−2 147 483 648 ... 2 147 483 647
long	4 u 8 (según SO)	Complement a 2	$-2^{31} \dots 2^{31} - 1$ (32-bit) $-2^{63} \dots 2^{63} - 1$ (64-bit Linux/Mac)
long long	8	Complement a 2	−9 223 372 036 854 775 808 ... 9 223 372 036 854 775 807
float	4	IEEE 754 (Simple precisió)	-3.4E38...3.4E38
double	8	IEEE 754 (Doble precisió)	-1.7E308...1.7E308
bool	1	Binari (0 o 1)	0 (false), 1 (true) (requereix <stdbool.h> en C99)
Punter (ej. int *)	4 u 8	Direcció de memòria	Depèn de la memòria del sistema

Per a saber quan ocupa un tipus. Ex: `printf("Un enter ocupa: %lu bytes\n", sizeof(int));`

Ex. Notació exponencial
 $3.4E8 = 3.4 \times 10^8$

Especificadors de format segons el tipus de dades.

Tipus	Especificador	Exemples	
		Sortida a pantalla	Entrada des de teclat
		<code>char c = 'a'; int a = 2; float x = 3.3f; double y = 2.345; bool b = true;</code>	
char	%c	<code>printf("%c", c);</code>	<code>scanf("%c", &c);</code>
int	%d	<code>printf("%d", a);</code>	<code>scanf("%d", &a);</code>
float	%f	<code>printf("%f", x);</code>	<code>scanf("%d", &x);</code>
double	%lf	<code>printf("%lf", y);</code>	<code>scanf("%d", &y);</code>
bool (es tracta como enter)	%d	<code>printf("%d", b);</code>	<code>scanf("%d", &b);</code>
cadena char	%s	Ho veurem més endavant	
punter (adreça)	%p	Ho veurem més endavant	

typedef: permet definir un nom alternatiu per a un tipus que ja existeix. Ho veurem m-es endavant amb structs

Conversió de tipus

- C es de tipat estàtic (s'ha de declarar en temps de compilació), i tipat feble (conversions implícites amb variables de diferent tipus).
- Amb variables de **diferent tipus**:

1. cast IMPLÍCIT

```
short a = 3; int b = 5;  
int i = a * b;
```

El short a (2 bytes) es converteix amb un cast IMPLÍCIT a int (4 bytes), es multipliquen els dos ints, i el int resultant s'assigna a i.

2. cast EXPLÍCIT

```
int a = 5, b = 2;  
float r = (float)a / b; // resultat 2.5, sense cast el resultat 2.0
```

El int a es converteix a float explícitament, es fa la divisió amb b que es converteix implícitament a float i el resultat s'assigna a r

2.3.2 Tipus de dades

Operació == amb float (pèrdua de precisió), idem amb double

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

```
#include <stdio.h>

int main(void) {
    float suma = 0.0f;    // Suma simple de decimales
    printf("0.1 + 0.2 = %.10f\n", 0.1f + 0.2f);
    printf("\nSumo mil veces 0.1f\n");
    for (int i = 0; i < 1000; i++) {
        suma += 0.1f;
    }

    // Comparamos el resultado con el valor esperado (100.0f)
    if (suma == 100.0f) {
        printf("El resultado es correcto: %f\n", suma);
    }
    else {
        printf("El resultado es incorrecto: %f\n", suma);
    }
}
```

Sortida a
pantalla

0.1 + 0.2 = 0.3000000119

Sumo mil veces 0.1f

El resultado es incorrecto: 99.999046

2.3.2 Tipus de dades

Operació == amb floats (pèrdua de precisió)

Solució a la pèrdua de precisió amb **epsilon**

```
#include <stdio.h>
#include <math.h>
```

```
int main(void) {
    float suma = 0.0f;
    float epsilon = 0.001f;
```



```
    printf("\nSumo mil veces 0.1, espero suma: 100.0\n");
    for (int i = 0; i < 1000; i++) {
        suma += 0.1f;
    }
    printf("suma vale: %.6f\n", suma);
    printf("fabs(suma - 100.0): %f\n", fabs(suma - 100.0f));
    printf("epsilon: %f\n", epsilon);
    // Comparamos el resultado con el valor esperado (100.0f) con un margen de error
    if (fabs(suma - 100.0f) < epsilon) {
        printf("El resultado es aproximadamente igual al esperado\n");
    }
    else {
        printf("La diferència es major a epsilon: %f\n", suma);
    }
    return 0;
}
```

Sortida a pantalla

```
Sumo mil veces 0.1, espero suma: 100.0
suma vale: 99.999046
fabs(suma - 100.0): 0.000954
epsilon: 0.001000
El resultado es aproximadamente igual al esperado
```

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

2.3.3 Expressions i operadors

Una expressió és una combinació de dades i operadors per calcular altres dades i així obtenir un resultat.

Ex.

`(a + b) / c`

`a % b` (resto de la divisió entre a i b)

`total * 0.21`

`a == b`

`(a + 1) != (b + 3)`

`suma = 3`

`suma += 3`

Una expressió és una combinació de dades i operadors per calcular altres dades i així obtenir un resultat.

Ex.

`(a + b) / c`

`a % b` (resto de la divisió entre a i b)

`total * 0.21`

`a == b`

`(a + 1) != (b + 3)`

`suma = 3`

`suma += 3`

- Si A i B són expressions.

`operadorUnari A` és una expressió.

`A operadorBinari B` és una expressió.

Ex.

`!(a > b)`

`(a > b) && (c < d)`

Taula d'operadors en C

Tipus	Valors (o literals)	Operadors
int	534	Unari: - Binaris: +, -, *, /, Módul: %
long long	40358LL	
float	5.0f	
double	5.0	
char	'c'	
bool	true	&& !

relacionals: ==, <, >, <=, >=, !=

assignació: +=, -=, *=, %=, /=, ...

Informació [sobre prioritat d'operadors](#).
Davant el dubte, useu parèntesis.

Un **literal** és un valor fix que s'escriu directament al codi font i que el programa no pot modificar durant la seva execució.
Per exemple, quan fem `int a = 534;`
`534` és un literal

Taula d'operadors lògics

A	B	A && B	A B	!A
true	true	true	true	false
true	false	false	true	
false	true	false	true	true
false	false	false	false	

Les **lleis de Morgan** permeten transformar una expressió lògica negada canviant AND (&&) → OR (||), OR (||) → AND (&&) i negant cada operand.

Per exemple:

`! (x > 0 && x < 10)` és equivalent a `x <= 0 || x >= 10`

Exemple amb operators

```
#include <stdio.h>
int main(void) {
    int x = 10, y = 35, r;
    // Operadores relacionales
    r = (x == y);           // 0: x no es igual a y
    r = (y != x * 2);       // 1: y es distinto de x * 2 (35 != 20)
    r = (y <= x);           // 0: y no es menor ni igual que x (35 <= 10 es falso)
    r = (x < y);            // 1: x es menor estricto que y (10 < 35)
    // Operadores aritméticos
    r = x * y;              // 350: multiplicación de x por y
    r = y / x;              // 3: división entera de y entre x
    r = y % x;              // 5: resto de la división de y entre x
    // Operadores lógicos
    r = (x < y && y < 50);  // 1: las dos condiciones son verdaderas
    r = (x > y || y < 50);  // 1: al menos una de las condiciones es verdadera
    r = !(x == y);          // 1: x == y es falso y ! lo convierte en verdadero

    return 0;
}
```

- Una sentència és un conjunt **d'expressions** que permeten executar una determinada acció (aka instrucció).
- Formen part sempre d'un bloc {<sentències>}
- Acaben en ;

- **Assignacions:** <identificador> = <expressió>;

```
int a = b * c;
```

- **Crides a mètodes.**

```
printf("a = %d\n", a);
```

- **Els comentaris a C.** // Comentari d'una línia,

```
/* Comentari de varies  
línies */
```

Ja vam parlar d'assignacions a la
pàgina [44](#) (variables)

Operador d'assignació

Permet donar valor a una variable `nomVariable = expressió;`

- S'avalua l'expressió i el valor resultat es posa com a nou contingut de `<nomVariable>`. Si la variable ja té un valor, aquest es sobreescriu!
- Si el tipus de la esquerra no coincideix amb el de la dreta, es produeix una conversió implícita. El programador també pot fer conversió explícita (veure [conversió de tipus](#))

```
int a = 1, b, c;  
float f;  
b = 2 + a;  
c = 4 + b;  
f = c;  
f += 2.1;
```

L'assignació `nomVariable += expressió`
equivale a `nomVariable = nomVariable + expressió`
Idem amb `-=` `*=` `/=` `%=`

2.3.3 Expressions i operadors

63

Operador ++ amb assignació (prefixo, sufix)

`nomVariable++` equival a `nomVariable = nomVariable + 1`

```
int a = 5, b = 6;  
a = ++b;  
System.out.println("a: " + a); //imprimeix a: 7  
System.out.println("b: " + b); //imprimeix b: 7
```

PREFIXO
++nomVariable, primer
incrementa i després assigna

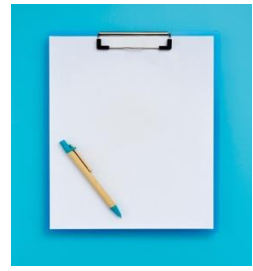
```
int a = 5, b = 6;  
a = b++;  
System.out.println("a: " + a); //imprimeix a: 6  
System.out.println("b: " + b); //imprimeix b: 7
```

SUFIXO
nomVariable++, primer
assigna i després
incrementa

Sense assignació, *no hi ha diferència entre* `++a;` i `a++;`

2.3.3 Expressions i operadors

Exercici



Tens un termòmetre de graus Fahrenheit, necessites per tant un conversor a temperatura Celsius. Escriu una funció anomenada `fahrenheit_a_celsius` que rebi com a paràmetre una temperatura en graus Fahrenheit (F) i retorni el seu equivalent en graus Celsius (C).

```
double fahrenheit_a_celsius(double fahrenheit)
```

Per a dur a terme la conversió, heu d'aplicar la fórmula següent: $C = (F - 32.0) * 5.0 / 9.0$

Pensa les variables, constants, expressions i sentències que necessites per resoldre'l

Escriu el programa en un únic fitxer: *fahrenheit.c*

Comprova que el teu conversor funciona (atenció a la imprecisió dels decimals!):

32.0 °F → 0.0 °C(Punt de congelació)

212.0 °F → 100.0 °C(Punt d'ebullició)

−40.0 °F → −40.0 °C(On les dues escales es creuen!)

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Àmbit de variables: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

2.3.4 Àmbit de variables: globals i locals

67

Àmbit (scope) de las variables

- **Variables globals.** Són aquelles que es declaren fora de tota funció, normalment abans del `main()`.
 - Són accessibles des de la seva declaració fins el final del programa.
 - Si no s'inicien, el valor per defecte és 0.
- **Variables locals.** Son aquelles que es declaren dintre d'una funció.
 - Són accessibles només dintre de la funció.

```
#include <stdio.h>

int edat; //v. global

int main(void)
{
    float altura = 1.85; //v. local
    edat = 20;
    printf("%d %f\n", edat, altura);

    return 0;
}
```

2.3.4 Àmbit de variables: globals i locals

68

Àmbit (scope) de las variables

- **Variables globals.** Són aquelles que es declaren fora de tota funció, normalment abans del `main()`.

- Són accessibles des del principi fins al final del programa.

- Si no s'inicien, el valor és 0.

- **Variables locals.** Son aquelles que es declaren dins d'una funció.

- Són accessibles només dins de la funció.

```
#include <stdio.h>
```

```
global
```

```
= 1.85; //v. local
```

```
f\n", edat, altura);
```

NO utilitzeu variables globals
És difícil controlar qui i com
les modifica

Contingut

2.1 Introducció

2.2 Estructura general d'un programa

2.2.1 Estructura d'un programa C

2.2.2 Funcions

2.2.3 Programa organitzat en mòduls

2.3 Elements del llenguatge C

2.3.1 Variables i constants

2.3.2 Tipus de dades

2.3.3 Expressions i operadors

2.3.4 Scope: variables locals i globals

2.3.5 Enums

2.4 Estructures de control avançat.

- Disseny sistemàtic de estructures de control iteratives

- Concepte de seqüència

- Esquemes de recorregut i cerca

- A C, el tipus `enum` serveix per a manipular un conjunt fix de constants (dies setmana, planetes, mesos de l'any, estats d'un joc, etc.). Internament cada constant es representa per un int (des de 0 en endavant)
- És un tipus creat per el programador que permet assignar noms llegibles a valors enters.
- Serveix per a fer el codi molt més llegible, net i fàcil de mantenir.
- Evita errors (per exemple, nom mal escrit al codi, i utilitzar un valor no permès)

```
enum NomTipusEnum {NOM_CONSTANT1, ... NOM_CONSTANTn};
```

```
enum DiaSetmana {DILLUNS, DIMARTS, DIMECRES, DIJOUS, DIVENDRES,  
DISSABTE, DIUMENGE};
```

Per convenció, s'utilitza CamelCase per al nom del tipus (ex. DiesSetmana, MesosAny)

```
#include <stdio.h>
#include <assert.h>
#include <stdbool.h>

enum DiesSetmana { DILLUNS, DIMARTS, DIMECRES,
                  DIJOUS, DIVENDRES, DISSABTE, DIUMENGE
                  };

/* Retorna true si és cap de setmana i false si és laborable */
bool esCapDeSetmana(enum DiesSetmana dia) {
    assert(dia >= DILLUNS && dia <= DIUMENGE);
    return dia == DISSABTE || dia == DIUMENGE;
}

int main(void) {
    assert(esCapDeSetmana(DISSABTE) == 1);
    assert(esCapDeSetmana(DIUMENGE) == 1);
    assert(esCapDeSetmana(DILLUNS) == 0);
    assert(esCapDeSetmana(DIMARTS) == 0);
    printf("TOTS els test passen\n");

    enum DiesSetmana dia;    dia = DIMARTS;
    if (esCapDeSetmana(dia)) {
        printf("Es cap de setmana.\n");
    }
    else {
        printf("Es laborable.\n");
    }
    return 0;
}
```

Exercici



Crea un programa en C que treballi amb una enum anomenat `ColorSemafor`.

1. Defineix l'enumeració `ColorSemafor` amb els valors `ROIG`, `GROC` i `VERD`.
2. Defineix una variable `color` de tipus `ColorSemafor`.
3. Assigna a la variable el valor `VERD`.
4. Mostra per pantalla el valor numèric de `color`.
5. Comprova amb un `if` si el color és `VERD`.
6. Si és `VERD` mostra per pantalla "Pots passar", sinó mostra "Para't"